CONFIDENTIAL designator

Red Hat
Enterprise Linux

API Remoting for `llama.cpp`: Near-Native GPU Speed in `macOS` Containers

Kevin Pouget, Jose Castillo Lema – January 31st, 2026
Red Hat Principal Software Engineer

FOSDEM - AI Plumbers track

Red Hat

Containers in macOS ???

But

, right ?

And

, no ???

So we need a **virtual machine** to bring containers to macOS

Red Hat

MacOS virtual container stack

podman desktop

libkrun

Libkrun is a tiny, embeddable VM runtime library that uses the host's native hypervisor APIs:

- macOS → Apple Hypervisor / Virtualization.framework
- Linux → KVM

MacOS virtual container stack

podman desktop

Podman AI Lab is the easiest way to work with LLMs on your local developer workstation.
Find a catalog of recipes, leverage a curated list of open source models, experiment and compare the model.

ramalama

Make working with AI boring through the use of OCI containers

```
ramalama pull llama3.2
ramalama run  llama3.2
ramalama bench llama3.2
```

CONFIDENTIAL designator

what about GPU acceleration ?

Red Hat

CONFIDENTIAL designator

what about GPU acceleration ?

podman desktop

- Linux container GPU acceleration is straightforward
  - just expose the required `/dev` files and userland libraries
- Virtual GPU acceleration **is fundamentally different**
  - **Full device passthrough?** not possible for the main display
  - **Hardware-assisted device passthrough?** not possible without … hardware assistance
  - **Virtual device emulation?** Not efficient for GPU acceleration
  - **Paravirtualized device?** Let's talk more about this!

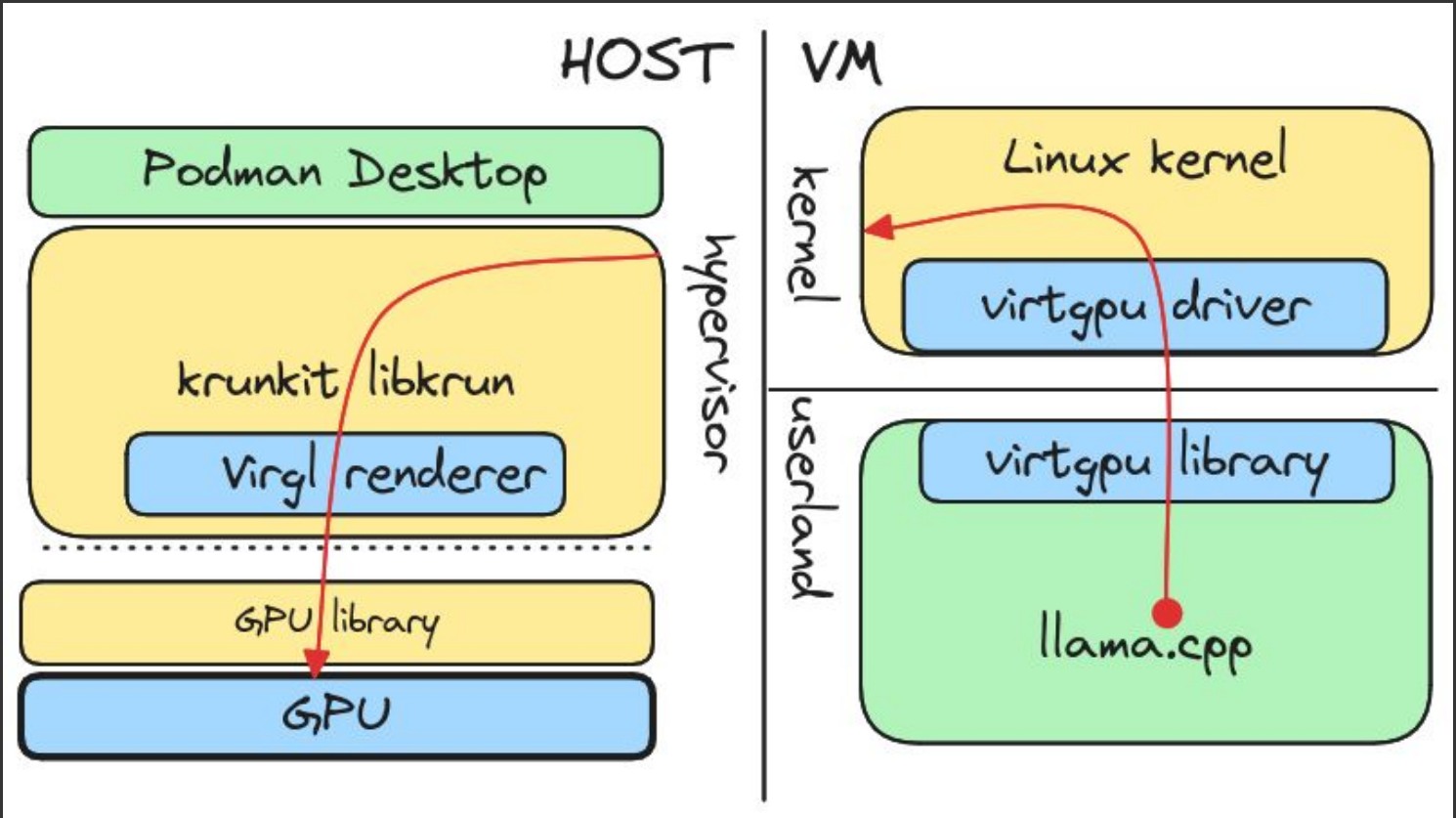
    (current solution for Vulkan acceleration & the API Remoting approach)

V0000000

Red Hat

MacOS Virtual GPU stack

podman desktop

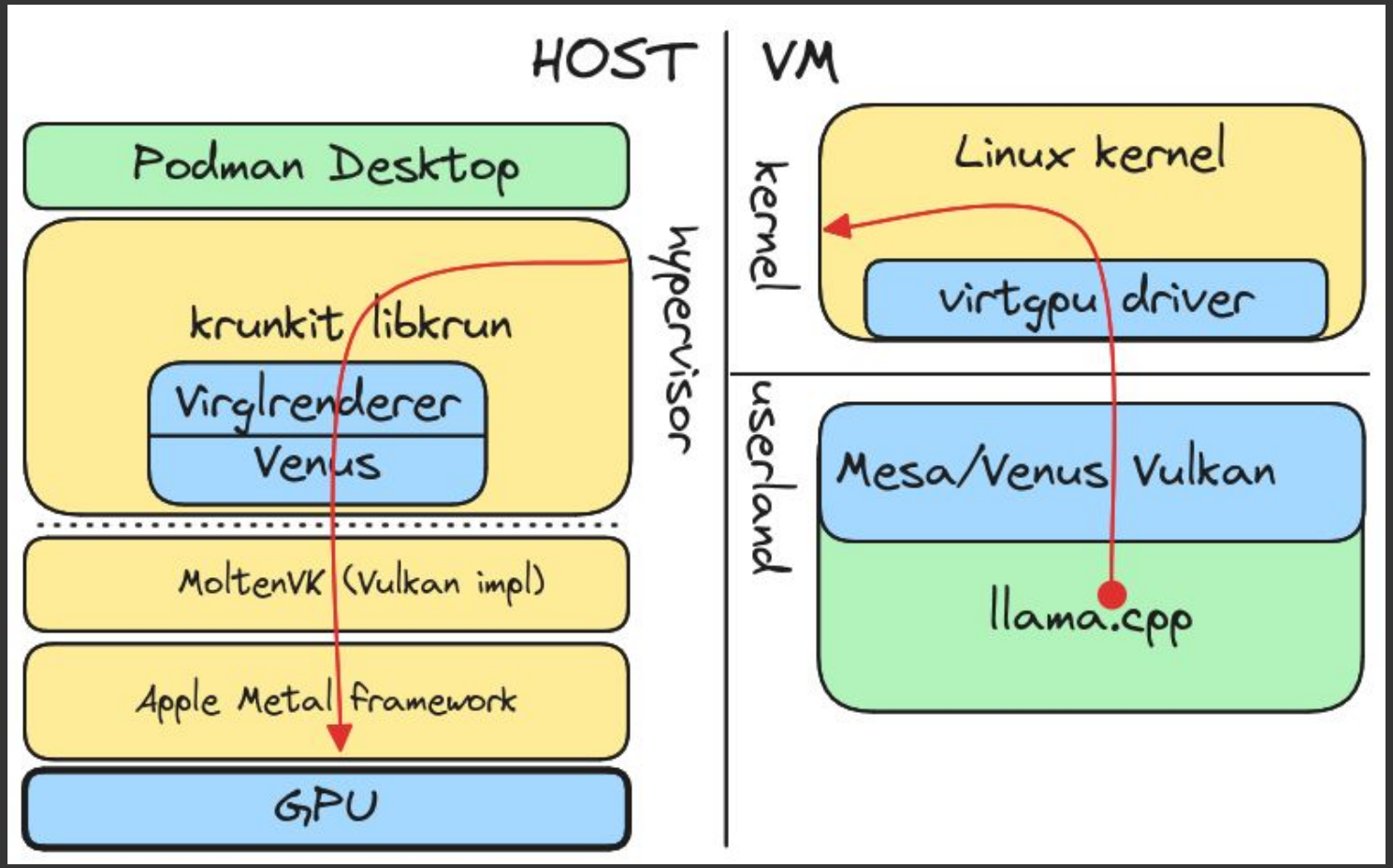


Red Hat

MacOS Virtual GPU stack

CONFIDENTIAL designator

podman desktop

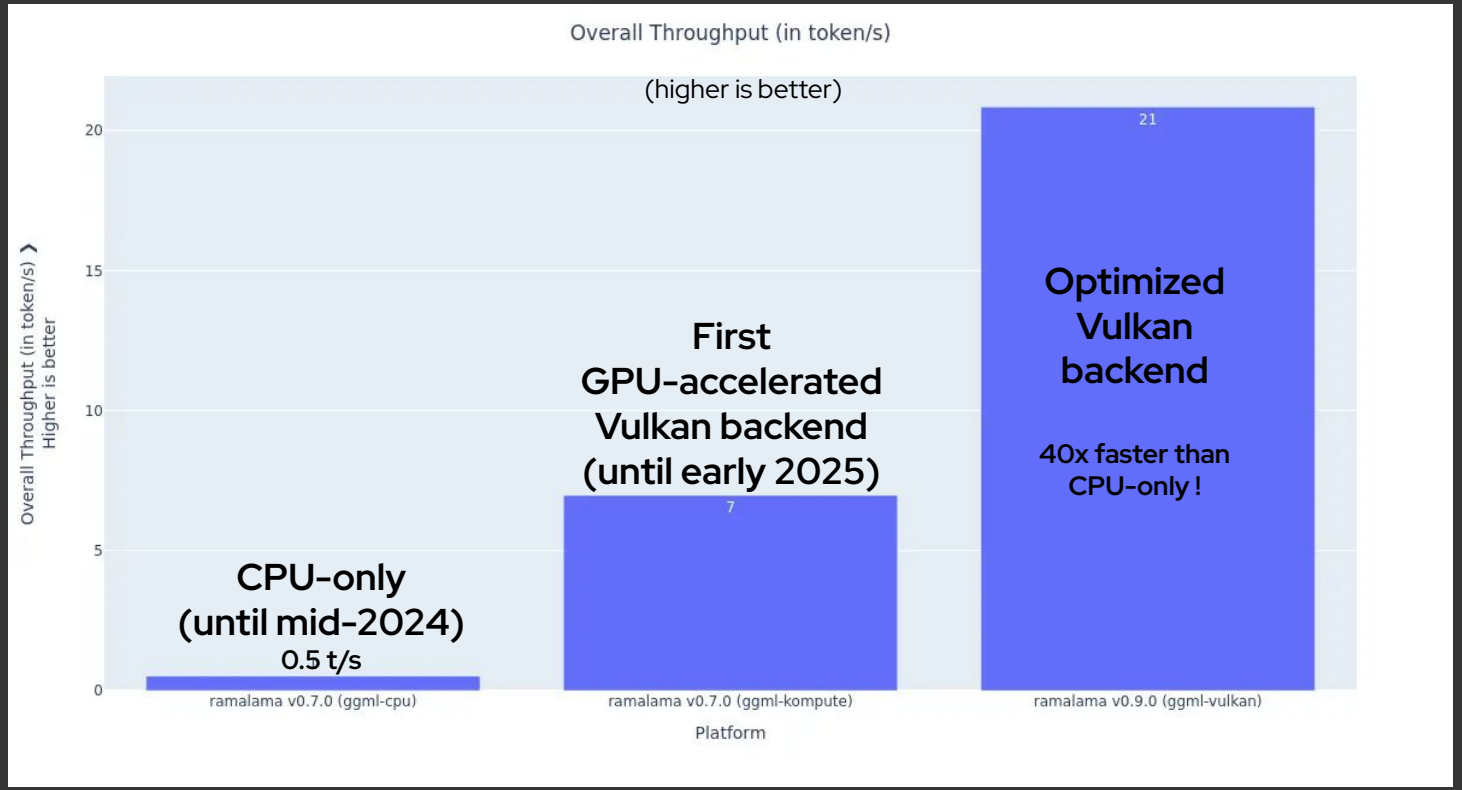


MacOS Virtual GPU stack performance

Red Hat

MacOS Virtual GPU stack performance

podman desktop



How we improved AI inference on macOS Podman containers

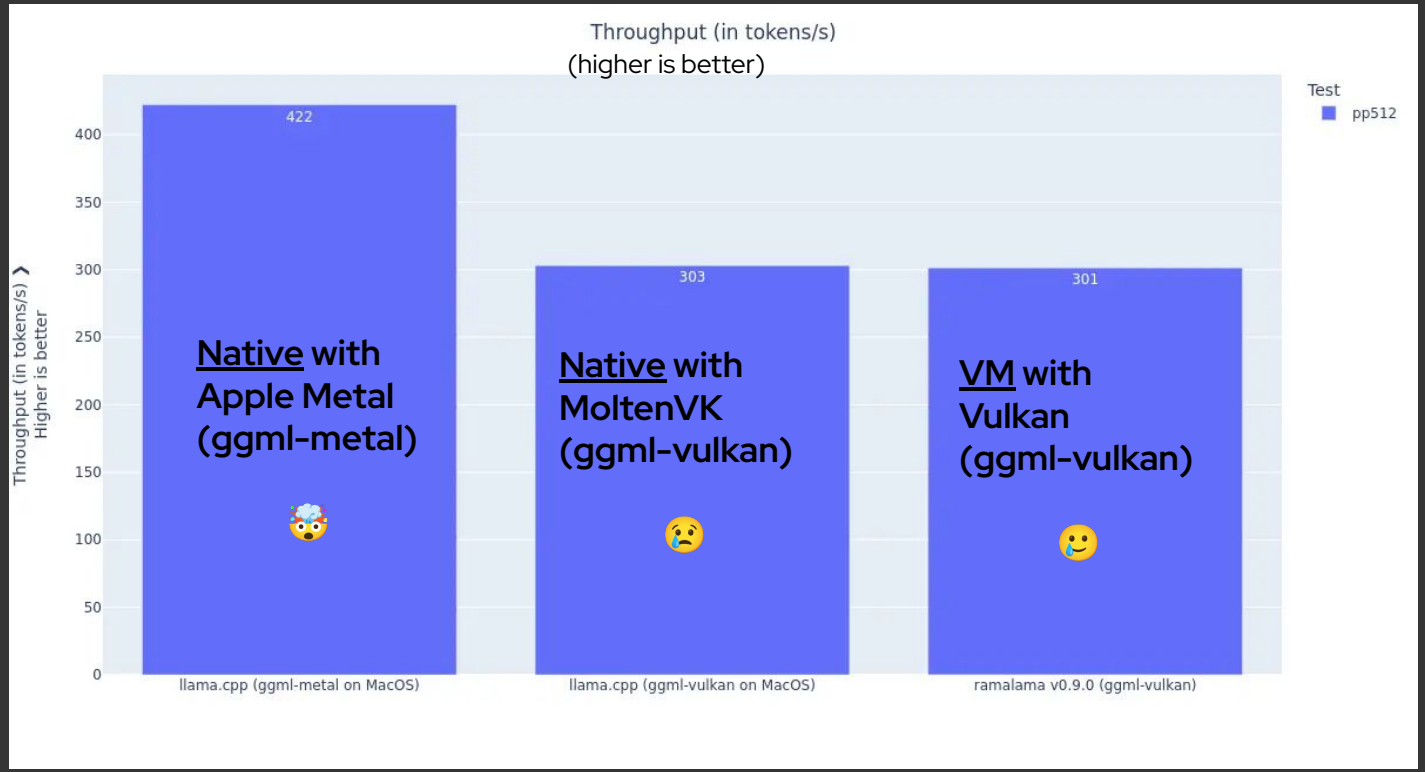
(developers.redhat.com)

Red Hat

MacOS Virtual GPU stack performance

CONFIDENTIAL designator

podman desktop



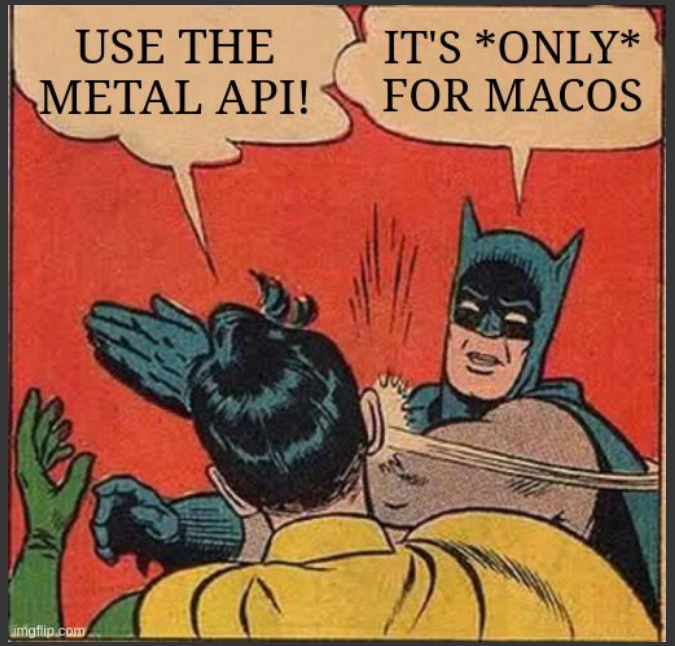
- Vulkan *MoltenVK* is the bottleneck :/
- New Mesa KosmicKrisp for MacOS Vulkan

Red Hat

How to improve the MacOS Virtual GPU stack performance ?

Red Hat

How to improve the macOS virtual GPU stack performance ?



- Deeply specific to MacOS
- Apple Swift
- legal aspects? 🤷‍♀️

Red Hat

How to improve the macOS virtual GPU stack performance ?

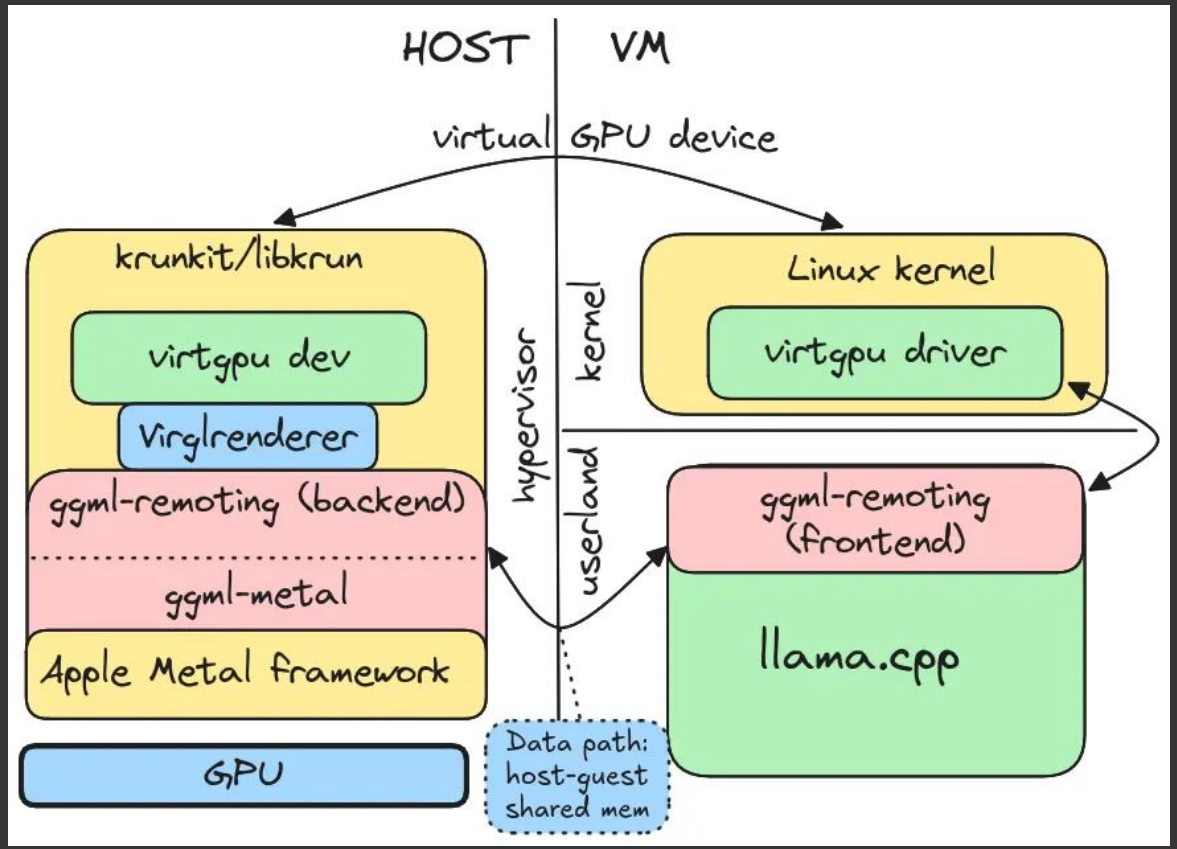
- GGML is the tensor library backend interface for `llama.cpp`
- Use GGML-Metal on MacOS
- Trigger the GGML API calls from the guest, execute them on the host

  ⇒ API Remoting!

Red Hat

API Remoting

podman desktop



Red Hat

API Remoting

- Intercept the API calls in the guest
- Forward the parameters to the host
- Call the actual API in the host
- Return the data back to the guest

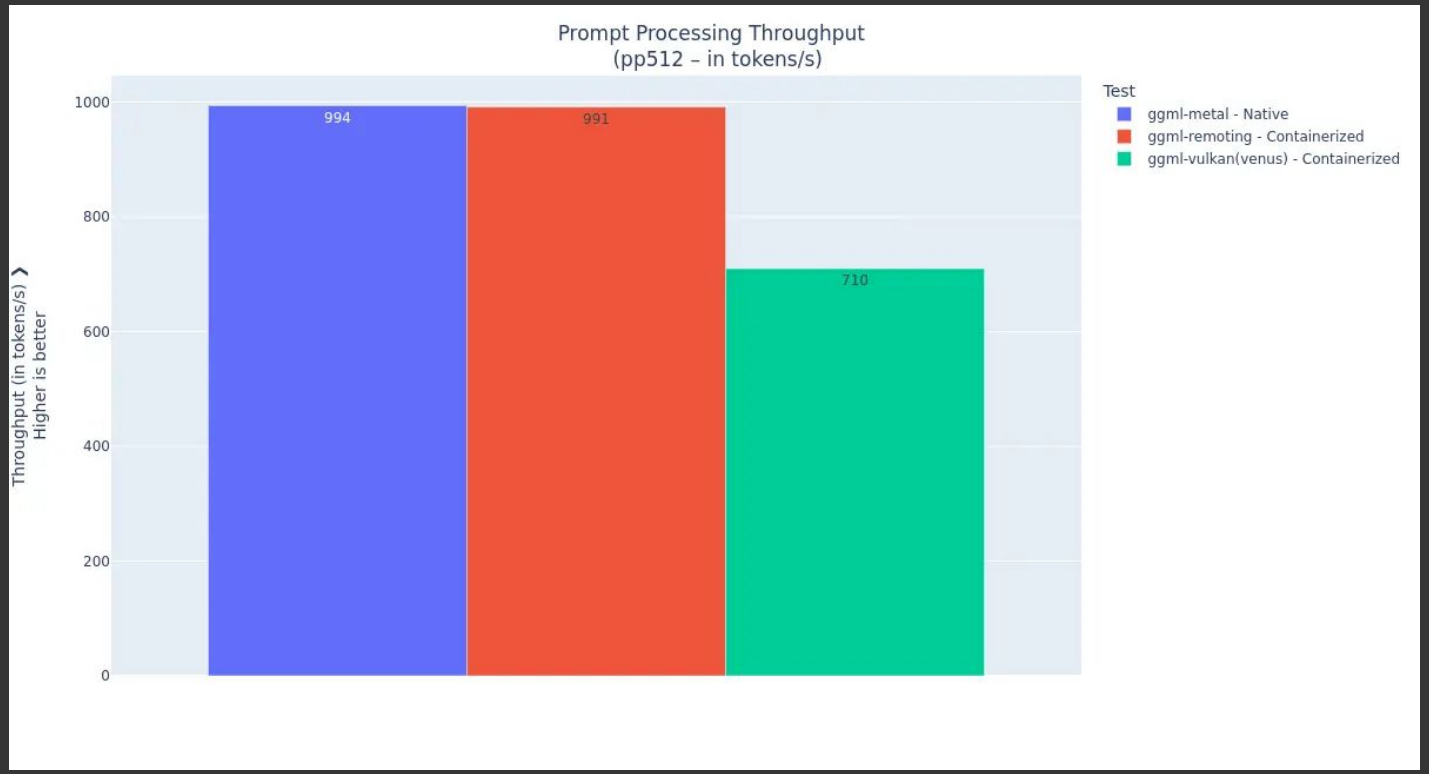
- 👎 Not a generic solution, specific to llama.cpp use case
- 👎 Pay the price of the hyper-calls latency + synchronization
- 👍 Use para-virtualization (host-guest shared memory pages) to avoid copying the data buffers

- Technique already used for the Vulkan-Venus communication

Red Hat

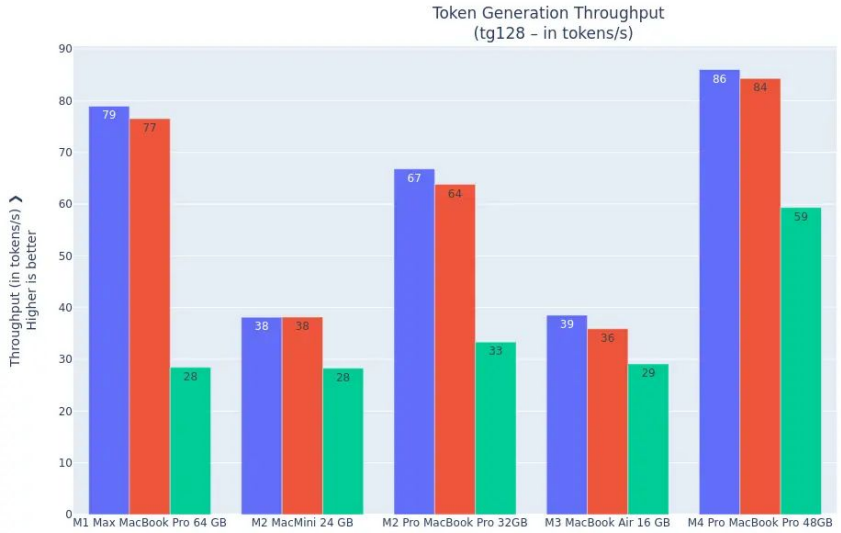
API Remoting POC

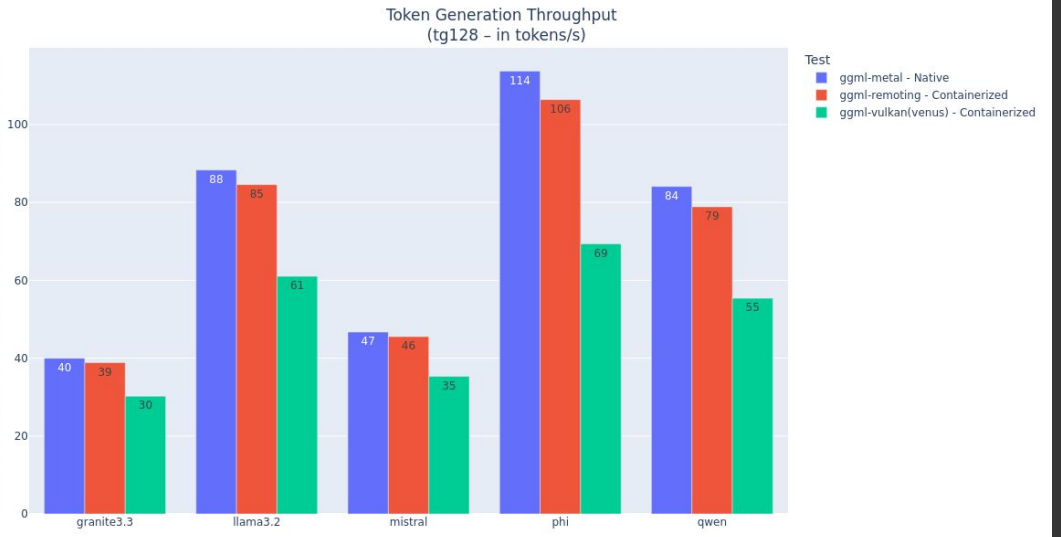
podman desktop

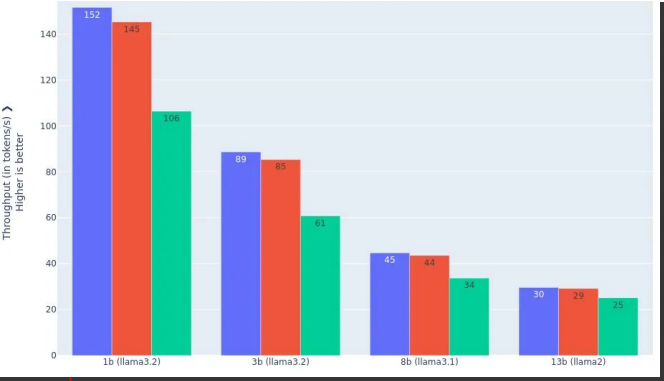


Red Hat

API Remoting POC







- Tested with different hardware
- Tested with different model families
- Tested with different model sizes

  ⇒ smaller models have more overhead (expected)

  - 151.59 t/s ⇔ 1 token every 6.6ms
  - 145.29 t/s ⇔ 1 token every 6.7ms

Red Hat

API Remoting Upstreaming status

podman desktop

`llama.cpp` (GGML frontend and backend)

- ggml: new backend for Virglrenderer API Remoting acceleration
  ⇒ merged earlier this week (Jan 26th) 🎉

`virglrenderer` (generic APIR component)

- Introduce an API Remoting trampoline component
  ⇒ review pending

Red Hat

API Remoting

- Reach native speed with MacOS llama.cpp container inference (developers.redhat.com)
  - Try with Podman Desktop ⇒ built an extension for easy testing
  - Try with Ramalama
    - Benchmark with Ramalama
  - Try it on Linux
    - with `krun` OCI container runtime
  - Try it on Windows
    - ~~kidding~~ 😛
    - well, not ready yet, still an early prototype

  - Integration in Podman Desktop
    - expected in the next quarter (Q2-2026)

podman desktop

Red Hat

API Remoting Security considerations

podman desktop

Key advantages

- the hypervisor (`krunkit`) runs with simple user privileges
- the backend and the GPU only execute trusted code
  - This eliminates risks coming from the execution of malicious kernels

Current limitations and considerations:

- The backend library currently lives within the hypervisor address space
  - a crash within the backend library could bring down the hypervisor.
  - the backends of multiple containers run in the same process (usual per-process GPU isolation not available)

Overall, we can say that this API remoting design isn't **multi-tenant safe.**

llama.cpp / Ramalama nightly CI

CONFIDENTIAL designator

entry_id	platform	containerized	version	pp_throughput	tg_throughput	llm_load_test_ttft	llm_load_test_itl
3	macos/llama_cpp/metal	False	b7770	415 tokens/s	39 token/s	1147 ms	25 ms
2	macos/llama_cpp/vulkan	False	b7770	241 tokens/s	28 token/s	2732 ms	46 ms
1	podman/ramalama	True	main-260115.1614	220 tokens/s	27 token/s	2684 ms	159 ms

Performed 12 KPI regression analyses over 3 entries x 4 KPIs. 0 KPIs didn't pass.

(link)

- Misc. bug: performance regression in llama-server (ggml-vulkan) #17033
- Eval bug: Vulkan not working on Intel GPU #17056

entry_id	platform	containerized	version	pp_throughput	tg_throughput	llm_load_test_ttft	llm_load_test_itl
4	macos/llama_cpp/metal	False	b6933	415 tokens/s	40 token/s	1133 ms	25 ms
3	macos/llama_cpp/vulkan	False	b6933	223 tokens/s	28 token/s	7459 ms	62 ms
1	podman/ramalama	True	main-251101.0607	223 tokens/s	28 token/s	3815 ms	42 ms
2	podman/ramalama/no-gpu	True	main-251101.0607	24 tokens/s	19 token/s	19192 ms	57 ms

Performed 16 KPI regression analyses over 4 entries x 4 KPIs. 5 KPIs didn't pass.

(link)

Red Hat
Enterprise Linux

API Remoting for `llama.cpp`: Near-Native GPU Speed in `macOS` Containers

Kevin Pouget, Jose Castillo Lema – January 31st, 2026
Red Hat Principal Software Engineer

FOSDEM - AI Plumbers track

Red Hat

API Remoting

podman deskt

Control path (guest side)

```
void ggml_device_get_memory(ggml_backend_dev_t dev, size_t * free, size_t * total) {

  // the encoder and decoder are shared memory buffers
  REMOTE_CALL_PREPARE(gpu, encoder, APIR_CMD_DEVICE_GET_MEMORY);

  apir_encode_ggml_dev(encoder, dev);

  REMOTE_CALL(gpu, encoder, decoder); // blocking call triggering the remote execution

  apir_decode_size_t(decoder, &free);
  apir_decode_size_t(decoder, &total);
}
```

API Remoting

CONFIDENTIAL designator

Data path initialization (buffer_type_alloc_buffer**)**

```
// guest side
virtgpu_shmem_create(gpu, size, &ctx.shmem)
apir_encode_virtgpu_shmem_res_id(encoder, ctx.shmem.res_id);
apir_encode_size_t(encoder, &size);

// host side
apir_decode_virtgpu_shmem_res_id(dec, &shmem_res_id);
shmem_ptr = ctx->iface->get_shmem_ptr(ctx->ctx_id, shmem_res_id); // to the hypervisor
// to the actual llama.cpp GGML backend
buffer = dev->iface.buffer_from_host_ptr(dev, shmem_ptr, size, max_tensor_size);
```

podman desktop

Data path transfers (guest side only)

```
// set_tensor
memcpy(shmem_ptr + offset, data, size);

// get_tensor
memcpy(data, shmem_ptr + offset, size);`
```

API Remoting

- Reach native speed with MacOS llama.cpp container inference (developers.redhat.com)
  - Try with Podman Desktop ⇒ built an extension for easy testing
  - Try with Ramalama:

podman desktop

```
# download and extract the tarball and enter its directory
bash ./update_krunkit.sh # update and re-sign the binaries
# restart the VM with the custom virglrenderer library
bash ./podman_start_machine.api_remoting.sh

export CONTAINERS_MACHINE_PROVIDER=libkrun
export IMG=quay.io/crcont/remoting:v0.16.0-apir…
ramalama run --image $IMG llama3.2
```

https://github.com/crc-org/llama.cpp/releases

API Remoting

- Reach native speed with MacOS llama.cpp container inference (developers.redhat.com)
  - Try with Podman Desktop ⇒ built an extension for easy testing
  - Try with Ramalama
  - Benchmark with Ramalama
    ⇒ reproduce the benchmark figures yourself!

podman desktop

```
ramalama bench --image $IMG llama3.2  # API Remoting performance
ramalama --nocontainer bench llama3.2 # native Metal performance
ramalama bench llama3.2               # Venus/Vulkan performance
```

https://github.com/crc-org/llama.cpp/releases

Red Hat

API Remoting

- Reach native speed with MacOS llama.cpp container inference (developers.redhat.com)
  - Try with Podman Desktop ⇒ built an extension for easy testing
  - Try with Ramalama
  - Benchmark with Ramalama
  - Try it on Linux

podman desktop

```
ramalama run --oci-runtime krun --image $IMG llama3.2
```

https://github.com/crc-org/llama.cpp/releases

Red Hat

API Remoting

- Reach native speed with MacOS llama.cpp container inference (developers.redhat.com)
  - Try with Podman Desktop ⇒ built an extension for easy testing
  - Try with Ramalama
  - Benchmark with Ramalama
  - Try it on Linux
  - Try it on Windows
    - kidding 😛

podman desktop